

Sorting

Comp Sci 214, Spring 2025

Don't forget to check in on Youhere!

Turning Chaos into Order

Sorting: take a sequence of elements in no particular order, and arrange them in a specific order.

- Take a linked list of numbers, and arrange them in decreasing order.
- Take an address book represented as a vector, and order it alphabetically by last name.
- Etc.

Why do we care?

- Making information easier to consume for humans
 - ▶ Every top n list ever (e.g., Billboard Top 40 list)
 - ▶ Finding an album in your music player
 - ▶ Etc.
- Making information easier to consume for algorithms
 - ▶ Binary search
 - ▶ Graph algorithms (in a few weeks)
 - ▶ Etc.

Turning Chaos into Order

Sorting is a classic problem in Computer Science.

Many different algorithms invented over decades to solve it.

- All solve the same problem, but diff. tradeoffs! (Cf. ADTs)
- Great illustrations of computational complexity!

Turning Chaos into Order

Sorting is a classic problem in Computer Science.

Many different algorithms invented over decades to solve it.

- All solve the same problem, but diff. tradeoffs! (Cf. ADTs)
- Great illustrations of computational complexity!

To start: comparing two simple algorithms

- *Selection sort*
- *Merge sort*

Then: some twists on sorting

- In-place sorting: *insertion sort*
- Sorting complex data: custom comparators, *radix sort*
- The classic sorting algorithm: *quicksort*

Bonus: more cool stuff we most likely won't get to

- Impossibly fast sorting: *counting sort*
- Comically bad sorts

Turning Chaos into Order

Sorting is a classic problem in Computer Science.

Many different algorithms invented over decades to solve it.

- All solve the same problem, but diff. tradeoffs! (Cf. ADTs)
- Great illustrations of computational complexity!

To start: comparing two simple algorithms

- *Selection sort*
- *Merge sort*

Code for all of them on Canvas!
(Plus some fun testing tech!)

Then: some twists on sorting

- In-place sorting: *insertion sort*
- Sorting complex data: custom comparators, *radix sort*
- The classic sorting algorithm: *quicksort*

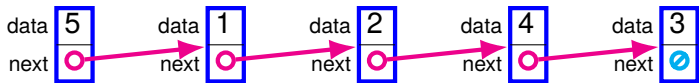
Bonus: more cool stuff we most likely won't get to

- Impossibly fast sorting: *counting sort*
- Comically bad sorts

Simple Sorting Algorithms

Selection Sort

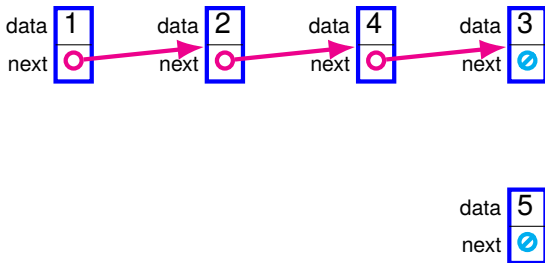
We'll demonstrate with linked lists; works with vectors too.



Find the largest element of the list (5).

Selection Sort

We'll demonstrate with linked lists; works with vectors too.

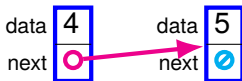
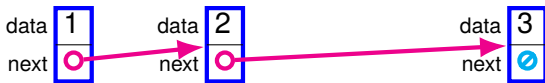


Find the largest element of the list (5).

Remove it from the input list, add it to the output list.

Selection Sort

We'll demonstrate with linked lists; works with vectors too.



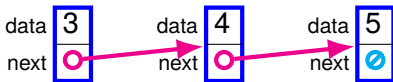
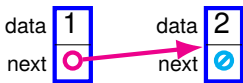
Find the largest element of the list (5).

Remove it from the input list, add it to the output list.

Then the next largest (4).

Selection Sort

We'll demonstrate with linked lists; works with vectors too.



Find the largest element of the list (5).

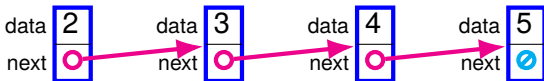
Remove it from the input list, add it to the output list.

Then the next largest (4).

And so on, building the output back-to-front.

Selection Sort

We'll demonstrate with linked lists; works with vectors too.



Find the largest element of the list (5).

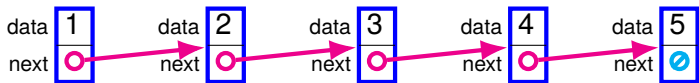
Remove it from the input list, add it to the output list.

Then the next largest (4).

And so on, building the output back-to-front.

Selection Sort

We'll demonstrate with linked lists; works with vectors too.



Find the largest element of the list (5).

Remove it from the input list, add it to the output list.

Then the next largest (4).

And so on, building the output back-to-front.

Selection Sort

```
def selection_sort (unsorted):  
    let sorted = None  
    while unsorted is not None:  
        let largest = find_largest(unsorted)  
        unsorted = remove(largest, unsorted)  
        sorted = cons(largest, sorted)  
    return sorted
```

`find_largest` and `remove` are what you would expect.
Iterating over a linked list from start to (worst case) finish.
See bonus slides for code and analysis.

We loop until the list is empty, removing one element each time.

QUIZ: How many iterations for a list of n elements?

A: 5

B: $\log n$

C: n

D: n^2

Selection Sort

```
def selection_sort (unsorted):  
    let sorted = None  
    while unsorted is not None:  
        let largest = find_largest(unsorted)  
        unsorted = remove(largest, unsorted)  
        sorted = cons(largest, sorted)  
    return sorted
```

find_largest and remove are what you would expect.
Iterating over a linked list from start to (worst case) finish.
See bonus slides for code and analysis.

We loop until the list is empty, removing one element each time.

QUIZ: How many iterations for a list of n elements?

A: 5

B: $\log n$

C: n

D: n^2

$$T_{\text{selection_sort}}(n) = n \times (T_{\text{is not}} + T_{\text{find_largest}}(n) + T_{\text{remove}}(n) + T_{\text{cons}})$$

Selection Sort

$$T_{\text{selection_sort}}(n) = n \times (T_{\text{is_not}} + T_{\text{find_largest}}(n) + T_{\text{remove}}(n) + T_{\text{cons}})$$

Selection Sort

$$\begin{aligned}T_{\text{selection_sort}}(n) &= n \times (T_{\text{is_not}} + T_{\text{find_largest}}(n) + T_{\text{remove}}(n) + T_{\text{cons}}) \\&= n \times (c_1 + T_{\text{find_largest}}(n) + T_{\text{remove}}(n))\end{aligned}$$

Selection Sort

$$\begin{aligned}T_{\text{selection_sort}}(n) &= n \times (T_{\text{is_not}} + T_{\text{find_largest}}(n) + T_{\text{remove}}(n) + T_{\text{cons}}) \\&= n \times (c_1 + T_{\text{find_largest}}(n) + T_{\text{remove}}(n)) \\&= n \times (c_1 + \mathcal{O}(n) + \mathcal{O}(n))\end{aligned}$$

Intuition: need to traverse the whole list in the worst case

Selection Sort

$$\begin{aligned}T_{\text{selection_sort}}(n) &= n \times (T_{\text{is_not}} + T_{\text{find_largest}}(n) + T_{\text{remove}}(n) + T_{\text{cons}}) \\&= n \times (c_1 + T_{\text{find_largest}}(n) + T_{\text{remove}}(n)) \\&= n \times (c_1 + \mathcal{O}(n) + \mathcal{O}(n)) \\&= n \times (c_1 + 2 \times \mathcal{O}(n))\end{aligned}$$

Selection Sort

$$\begin{aligned}T_{\text{selection_sort}}(n) &= n \times (T_{\text{is_not}} + T_{\text{find_largest}}(n) + T_{\text{remove}}(n) + T_{\text{cons}}) \\&= n \times (c_1 + T_{\text{find_largest}}(n) + T_{\text{remove}}(n)) \\&= n \times (c_1 + \mathcal{O}(n) + \mathcal{O}(n)) \\&= n \times (c_1 + 2 \times \mathcal{O}(n)) \\&= n \times (c_1 + \mathcal{O}(n))\end{aligned}$$

$$\mathcal{O}(c \times f(n)) = \mathcal{O}(f(n))$$

Multiplying by constants doesn't matter.

Selection Sort

$$\begin{aligned}T_{\text{selection_sort}}(n) &= n \times (T_{\text{is_not}} + T_{\text{find_largest}}(n) + T_{\text{remove}}(n) + T_{\text{cons}}) \\&= n \times (c_1 + T_{\text{find_largest}}(n) + T_{\text{remove}}(n)) \\&= n \times (c_1 + \mathcal{O}(n) + \mathcal{O}(n)) \\&= n \times (c_1 + 2 \times \mathcal{O}(n)) \\&= n \times (c_1 + \mathcal{O}(n)) \\&= n \times \mathcal{O}(n)\end{aligned}$$

$$\mathcal{O}(f(n) + c) = \mathcal{O}(f(n))$$

Adding constants doesn't matter.

Selection Sort

$$\begin{aligned}T_{\text{selection_sort}}(n) &= n \times (T_{\text{is_not}} + T_{\text{find_largest}}(n) + T_{\text{remove}}(n) + T_{\text{cons}}) \\&= n \times (c_1 + T_{\text{find_largest}}(n) + T_{\text{remove}}(n)) \\&= n \times (c_1 + \mathcal{O}(n) + \mathcal{O}(n)) \\&= n \times (c_1 + 2 \times \mathcal{O}(n)) \\&= n \times (c_1 + \mathcal{O}(n)) \\&= n \times \mathcal{O}(n) \\&= \mathcal{O}(n^2)\end{aligned}$$

$$n \times n = n^2$$

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

4	1	7
---	---	---

2	3	0
---	---	---

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Step 2: Recursively sort each sub-list

4	1	7
---	---	---

2	3	0
---	---	---

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Step 2: Recursively sort each sub-list

1	4	7
---	---	---

0	2	3
---	---	---

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Step 2: Recursively sort each sub-list

1	4	7
---	---	---

0	2	3
---	---	---

Step 3: Merge the two, picking the smallest element from either at each step

--	--	--	--	--	--

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Step 2: Recursively sort each sub-list

1	4	7
---	---	---

0	2	3
---	---	---

Step 3: Merge the two, picking the smallest element from either at each step

0					
---	--	--	--	--	--

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Step 2: Recursively sort each sub-list

1	4	7
---	---	---

0	2	3
---	---	---

Step 3: Merge the two, picking the smallest element from either at each step

0	1				
---	---	--	--	--	--

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Step 2: Recursively sort each sub-list

1	4	7
---	---	---

0	2	3
---	---	---

Step 3: Merge the two, picking the smallest element from either at each step

0	1	2			
---	---	---	--	--	--

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Step 2: Recursively sort each sub-list

1	4	7
---	---	---

0	2	3
---	---	---

Step 3: Merge the two, picking the smallest element from either at each step

0	1	2	3		
---	---	---	---	--	--

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Step 2: Recursively sort each sub-list

1	4	7
---	---	---

0	2	3
---	---	---

Step 3: Merge the two, picking the smallest element from either at each step

0	1	2	3	4	
---	---	---	---	---	--

Merge Sort

(Vector notation for clarity; works even better on linked lists!)

0	1	2	3	4	5
4	2	1	3	7	0

Step 1: Split into two sub-lists (even indices and odd indices)

Step 2: Recursively sort each sub-list

1	4	7
---	---	---

0	2	3
---	---	---

Step 3: Merge the two, picking the smallest element from either at each step

0	1	2	3	4	7
---	---	---	---	---	---

Merge Sort

```
# : List[Number] -> List[Number]
def merge_sort(lst):
    # Base cases
    if lst is None or lst.next is None: return lst
    # Step 1: split into two sub-lists
    let o = odds(lst)
    let e = evens(lst)
    # Step 2: recursively sort each sub-list
    o = merge_sort(o)
    e = merge_sort(e)
    # Step 3: merge the two sub-lists
    return merge(o, e)
```

Merge Sort

```
# : List[Number] -> List[Number]
def merge_sort(lst):
    # Base cases
    if lst is None or lst.next is None: return lst
    # Step 1: split into two sub-lists
    let o = odds(lst)
    let e = evens(lst)
    # Step 2: recursively sort each sub-list
    o = merge_sort(o)
    e = merge_sort(e)
    # Step 3: merge the two sub-lists
    return merge(o, e)
```

$$T_{\text{merge_sort}}(1) = T_{\text{is}} + T_{\text{.next}} + T_{\text{is}}$$

$$\begin{aligned} T_{\text{merge_sort}}(n) = & T_{\text{is}} + T_{\text{.next}} + T_{\text{is}} + T_{\text{odds}}(n) + T_{\text{evens}}(n) \\ & + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + T_{\text{merge}}(n) \end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = T_{\text{is}} + T_{\text{.next}} + T_{\text{is}}$$

$$\begin{aligned} T_{\text{merge_sort}}(n) = & T_{\text{is}} + T_{\text{.next}} + T_{\text{is}} + T_{\text{odds}}(n) + T_{\text{evens}}(n) \\ & + T_{\text{merge_sort}}(\frac{n}{2}) + T_{\text{merge_sort}}(\frac{n}{2}) + T_{\text{merge}}(n) \end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned} T_{\text{merge_sort}}(n) = c_1 + T_{\text{odds}}(n) + T_{\text{evens}}(n) \\ + T_{\text{merge_sort}}(\frac{n}{2}) + T_{\text{merge_sort}}(\frac{n}{2}) + T_{\text{merge}}(n) \end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned} T_{\text{merge_sort}}(n) &= c_1 + T_{\text{odds}}(n) + T_{\text{evens}}(n) \\ &\quad + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + T_{\text{merge}}(n) \\ &= c_1 + \mathcal{O}(n) + \mathcal{O}(n) \\ &\quad + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + \mathcal{O}(n) \end{aligned}$$

Intuition: need to traverse the whole list(s)

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned} T_{\text{merge_sort}}(n) &= c_1 + T_{\text{odds}}(n) + T_{\text{evens}}(n) \\ &\quad + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + T_{\text{merge}}(n) \\ &= c_1 + \mathcal{O}(n) + \mathcal{O}(n) \\ &\quad + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + T_{\text{merge_sort}}\left(\frac{n}{2}\right) + \mathcal{O}(n) \\ &= \mathcal{O}(n) + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \end{aligned}$$

Constant multiplication, but not of $\mathcal{O}(f(n))$! Can't remove.

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$T_{\text{merge_sort}}(n) = n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right)$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned} T_{\text{merge_sort}}(n) &= n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \\ &= n + 2 \times \left(\frac{n}{2} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right)\right) \end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned} T_{\text{merge_sort}}(n) &= n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \\ &= n + 2 \times \left(\frac{n}{2} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right)\right) \\ &= n + 2 \times \frac{n}{2} + 4 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right) \end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned} T_{\text{merge_sort}}(n) &= n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \\ &= n + 2 \times \left(\frac{n}{2} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right)\right) \\ &= n + 2 \times \frac{n}{2} + 4 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right) \\ &= n + 2 \times \frac{n}{2} + 4 \times \left(\frac{n}{4} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\right) \end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned} T_{\text{merge_sort}}(n) &= n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \\ &= n + 2 \times \left(\frac{n}{2} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right)\right) \\ &= n + 2 \times \frac{n}{2} + 4 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right) \\ &= n + 2 \times \frac{n}{2} + 4 \times \left(\frac{n}{4} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\right) \\ &= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + 8 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right) \end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned}T_{\text{merge_sort}}(n) &= n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \\&= n + 2 \times \left(\frac{n}{2} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \left(\frac{n}{4} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + 8 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right) \\&\dots \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + \dots + n \times T_{\text{merge_sort}}(1)\end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned}T_{\text{merge_sort}}(n) &= n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \\&= n + 2 \times \left(\frac{n}{2} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \left(\frac{n}{4} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + 8 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right) \\&\dots \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + \dots + n \times T_{\text{merge_sort}}(1) \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + \dots + n \times c_1\end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned}T_{\text{merge_sort}}(n) &= n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \\&= n + 2 \times \left(\frac{n}{2} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \left(\frac{n}{4} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + 8 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\end{aligned}$$

...

$$\begin{aligned}&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + \dots + n \times T_{\text{merge_sort}}(1) \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + \dots + n \times c_1 \\&= n + n + n + \dots + n \times c_1\end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned}T_{\text{merge_sort}}(n) &= n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \\&= n + 2 \times \left(\frac{n}{2} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \left(\frac{n}{4} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + 8 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\end{aligned}$$

...

$$\begin{aligned}&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + \dots + n \times T_{\text{merge_sort}}(1) \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + \dots + n \times c_1 \\&= n + n + n + \dots + n \times c_1 \\&= \log n \times n + n \times c_1\end{aligned}$$

Merge Sort

$$T_{\text{merge_sort}}(1) = c_1$$

$$\begin{aligned}T_{\text{merge_sort}}(n) &= n + 2 \times T_{\text{merge_sort}}\left(\frac{n}{2}\right) \\&= n + 2 \times \left(\frac{n}{2} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times T_{\text{merge_sort}}\left(\frac{n}{4}\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \left(\frac{n}{4} + 2 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\right) \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + 8 \times T_{\text{merge_sort}}\left(\frac{n}{8}\right)\end{aligned}$$

...

$$\begin{aligned}&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + \dots + n \times T_{\text{merge_sort}}(1) \\&= n + 2 \times \frac{n}{2} + 4 \times \frac{n}{4} + \dots + n \times c_1 \\&= n + n + n + \dots + n \times c_1 \\&= \log n \times n + n \times c_1 \\&= \mathcal{O}(n \log n)\end{aligned}$$

Selection Sort versus Merge Sort

Selection sort takes $\mathcal{O}(n^2)$. Merge sort takes $\mathcal{O}(n \log n)$.

	n	n^2	$n \log n$
	16	256	64
	32	1024	160
	64	4096	384
	256	65536	2048
$(10^6 \approx)$	2^{20}	1.1×10^{12}	2.1×10^7

$n \log n$ grows faster than n , but **much** slower than n^2 .

Selection Sort versus Merge Sort

Selection sort takes $\mathcal{O}(n^2)$. Merge sort takes $\mathcal{O}(n \log n)$.

	n	n^2	$1\,000\,000 \times n \log n$
	16	256	64\,000\,000
	32	1024	160\,000\,000
	64	4\,096	384\,000\,000
	256	65\,536	2\,048\,000\,000
$(10^6 \approx)$	2^{20}	1.1×10^{12}	2.1×10^{13}

$n \log n$ grows faster than n , but **much** slower than n^2 .

Selection Sort versus Merge Sort

Selection sort takes $\mathcal{O}(n^2)$. Merge sort takes $\mathcal{O}(n \log n)$.

	n	n^2	$1\,000\,000 \times n \log n$
	16	256	64 000 000
	32	1024	160 000 000
	64	4 096	384 000 000
	256	65 536	2 048 000 000
$(10^6 \approx)$	2^{20}	1.1×10^{12}	2.1×10^{13}
$(3.6 \times 10^7 \approx)$	2^{25}	1.1×10^{15}	8.4×10^{14}

$n \log n$ grows faster than n , but **much** slower than n^2 .

Selection Sort versus Merge Sort

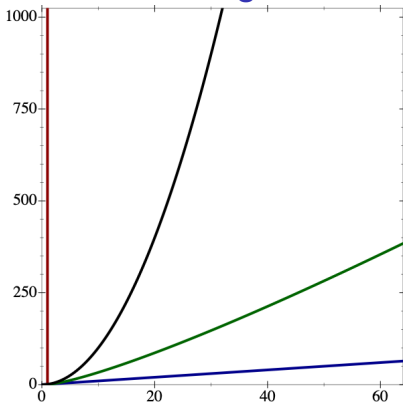
Selection sort takes $\mathcal{O}(n^2)$. Merge sort takes $\mathcal{O}(n \log n)$.

	n	n^2	$1\,000\,000 \times n \log n$
	16	256	64 000 000
	32	1024	160 000 000
	64	4 096	384 000 000
	256	65 536	2 048 000 000
$(10^6 \approx)$	2^{20}	1.1×10^{12}	2.1×10^{13}
$(3.6 \times 10^7 \approx)$	2^{25}	1.1×10^{15}	8.4×10^{14}

Dawson's First Law: $\mathcal{O}(n^2)$ is the “sweet spot” of badly scaling algorithms: fast enough to make it into production, but slow enough to make things fall down once it gets there.

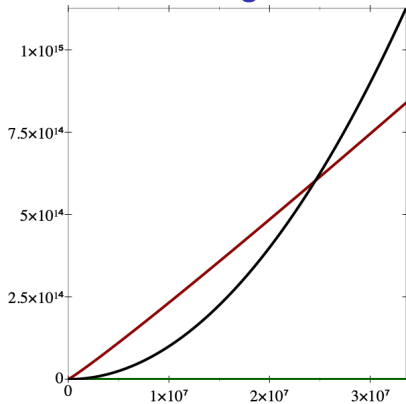
<https://accidentallyquadratic.tumblr.com>

Selection Sort versus Merge Sort



red	$1\,000\,000 \times n \log n$
black	n^2
green	$n \log n$
blue	n

Selection Sort versus Merge Sort



red	$1\,000\,000 \times n \log n$
black	n^2
green	$n \log n$
blue	n

The $n \log n$ Barrier

So far, merge sort looks pretty good at $\mathcal{O}(n \log n)$!

But is $\mathcal{O}(n \log n)$ the best we can do?

The $n \log n$ Barrier

So far, merge sort looks pretty good at $\mathcal{O}(n \log n)$!

But is $\mathcal{O}(n \log n)$ the best we can do?

For sorts that rely on *comparisons* (i.e., $a < b$), **yes**.

- Mathematically impossible to beat. We won't do the proof.

The $n \log n$ Barrier

So far, merge sort looks pretty good at $\mathcal{O}(n \log n)$!

But is $\mathcal{O}(n \log n)$ the best we can do?

For sorts that rely on *comparisons* (i.e., $a < b$), **yes**.

- Mathematically impossible to beat. We won't do the proof.

But there's tricks for sorting without using comparisons!

With those, it's possible to sort in $\mathcal{O}(n)$!

Only applicable under certain conditions, but great when it is!

Details in bonus slides.

Stop! Question time!

Before we move on to in-place sorting...

Anything unclear so far?

Anything I missed?

In-Place Sorting

In-Place Sorting

In-place sorting: sorting a vector* by modifying it, without relying on separate auxiliary storage.

* needs $\mathcal{O}(1)$ get/set_nth, so lists are right out.

Contrast **out-of-place** sorting

- Merge sort creates two new sub-lists each step
- Also: separate list for the output!

In-Place Sorting

In-place sorting: sorting a vector* by modifying it, without relying on separate auxiliary storage.

* needs $\mathcal{O}(1)$ get/set_nth, so lists are right out.

Contrast **out-of-place** sorting

- Merge sort creates two new sub-lists each step
- Also: separate list for the output!

Why are in-place algorithms useful?

- Use less space: no need to allocate extra data structures
- Useful for large inputs! (or small computers)

In-Place Sorting

In-place sorting: sorting a vector* by modifying it, without relying on separate auxiliary storage.

* needs $\mathcal{O}(1)$ get/set_nth, so lists are right out.

Contrast **out-of-place** sorting

- Merge sort creates two new sub-lists each step
- Also: separate list for the output!

Why are in-place algorithms useful?

- Use less space: no need to allocate extra data structures
- Useful for large inputs! (or small computers)

Why are out-of-place algorithms useful?

- Don't modify their inputs $\rightarrow$ can still use the original
- Often simpler $\rightarrow$ less risk of bugs!
- It's not the 60s anymore

Both are useful in different contexts: choose wisely!

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

3	5	1	2	4
---	---	---	---	---

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

3	5	1	2	4
---	---	---	---	---

First 1 element is trivially sorted.

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

3	5	1	2	4
---	---	---	---	---

2nd element is $\geq$ 1st; first 2 elements are sorted.

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

3	5	1	2	4
---	---	---	---	---

3rd element is not $\geq$ 2nd; swap!

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

3	1	5	2	4
---	---	---	---	---

New 2nd element is not $\geq$ 1st; swap again!

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

1	3	5	2	4
---	---	---	---	---

First 3 elements are sorted.

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

1	3	5	2	4
---	---	---	---	---

4th element is not $\geq$ 3rd; swap!

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

1	3	2	5	4
---	---	---	---	---

New 3rd element is not $\geq$ 2nd; swap!

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

1	2	3	5	4
---	---	---	---	---

New 2nd element is $\geq$ 1st; first 4 elements are sorted.

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

1	2	3	5	4
---	---	---	---	---

5th element is not $\geq$ 4th; swap!

In-Place Sorting: Insertion Sort

Let's illustrate with a simple algorithm: *insertion sort*

Idea: keep the start of the vector sorted, and gradually “insert” later elements in their correct positions by *swapping*.

1	2	3	4	5
---	---	---	---	---

New 4th element is $\geq$ 3rd; the entire vector is sorted!

Fun fact: you may have invented this yourself to sort a hand of cards while playing bridge, spades, crazy eights, etc.

In-Place Sorting: Insertion Sort

```
def insertion_sort (vec: VecC[AnyC]) -> NoneC:  
    let i = 1  
    while i < vec.len():  
        let j = i  
        while j > 0 and vec[j-1] > vec[j]:  
            # swap vec[j] and vec[j-1]  
            let tmp = vec[j]  
            vec[j] = vec[j-1]  
            vec[j-1] = tmp  
            j = j - 1  
        i = i + 1
```

i is the length of the sorted prefix.

j is the index of the element we're swapping.

First three lines of the inner loop do the swapping.

- Pattern (use of temporary variable) you'll see often!

Modifies the input vector! (in-place)

In-Place Sorting: Insertion Sort

```
def insertion_sort (vec: VecC[AnyC]) -> NoneC:
  let i = 1
  while i < vec.len():
    let j = i
    while j > 0 and vec[j-1] > vec[j]:
      # swap vec[j] and vec[j-1]
      let tmp = vec[j]
      vec[j] = vec[j-1]
      vec[j-1] = tmp
      j = j - 1
    i = i + 1
```

Inner loop: j goes from i to 0 in the worst case.

- I.e., from 1 to 0, 2 to 0, 3 to 0, ..., n to 0.
- $= 1 + 2 + \dots + n = \mathcal{O}(n^2)$
- I.e., not something you actually want to use.
 - ▶ But there are better in-place algorithms! (Stay tuned.)

Stop! Question time!

Before we move on to sorting complex data...

Anything unclear so far?

Anything I missed?

Sorting Complex Data

I am not a number, I am a free man!

We've seen how to sort numbers: simplest case of sorting.

But often, what we want to sort is more complex data!

I am not a number, I am a free man!

We've seen how to sort numbers: simplest case of sorting.

But often, what we want to sort is more complex data!

Example: sorting restaurants by their rating on Yelp.

Restaurant	★	\$
Aloha Eats	4	2
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
Maison Parisienne	4	3
Fons Empanadas	5	2
Del Seoul	4.5	2
The Weiner's Circle	3.5	1



Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
Aloha Eats	4	2
Cafe Luigi	4	1
Maison Parisienne	4	3
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1

I am not a number, I am a free man!

We've seen how to sort numbers: simplest case of sorting.

But often, what we want to sort is more complex data!

Example: sorting restaurants by their rating on Yelp.

Restaurant	★	\$
Aloha Eats	4	2
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
Maison Parisienne	4	3
Fons Empanadas	5	2
Del Seoul	4.5	2
The Weiner's Circle	3.5	1



Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
Aloha Eats	4	2
Cafe Luigi	4	1
Maison Parisienne	4	3
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1

Need to use a custom comparison function, e.g.:

```
lambda r1, r2: r1.stars > r2.stars
```

instead of just >

Insertion Sort with Custom Comparator

```
def insertion_sort_custom (vec, cmp): # extra argument!
    let i = 1
    while i < vec.len():
        let j = i
        while j > 0 and \
            cmp(vec[j-1], vec[j]): # the only change!
            let tmp = vec[j]
            vec[j] = vec[j-1]
            vec[j-1] = tmp
            j = j - 1
        i = i + 1
```

Used like this:

```
inser...(restaurants, lambda r1, r2: r1.stars > r2.stars)
```

Other sorting algorithms can be adapted in similar ways.

Bonus: can be used to sort in increasing/decreasing order, too!

Sorting Using Multiple Criteria

What if we want a good meal, but as cheaply as possible?

- Order by rating (decreasing), *then* by price (increasing)!

Restaurant	★	\$
Aloha Eats	4	2
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
Maison Parisienne	4	3
Fons Empanadas	5	2
Del Seoul	4.5	2
The Weiner's Circle	3.5	1



Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
<i>Cafe Luigi</i>	<i>4</i>	<i>1</i>
<i>Aloha Eats</i>	<i>4</i>	<i>2</i>
<i>Maison Parisienne</i>	<i>4</i>	<i>3</i>
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1

How can we do that?

Sorting Using Multiple Criteria

What if we want a good meal, but as cheaply as possible?

- Order by rating (decreasing), *then* by price (increasing)!

Restaurant	★	\$
Aloha Eats	4	2
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
Maison Parisienne	4	3
Fons Empanadas	5	2
Del Seoul	4.5	2
The Weiner's Circle	3.5	1



Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
<i>Cafe Luigi</i>	<i>4</i>	<i>1</i>
<i>Aloha Eats</i>	<i>4</i>	<i>2</i>
<i>Maison Parisienne</i>	<i>4</i>	<i>3</i>
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1

First attempt: a comparison function that looks at both

- Compares ratings, uses price as a tiebreaker

```
def better_deal_than (r1, r2):  
    if r1.stars > r2.stars:           return True  
    else if r1.stars == r2.stars  
        and r1.price < r2.price:     return True  
    else:                             return False
```

Sorting Using Multiple Criteria

What if we want a good meal, but as cheaply as possible?

- Order by rating (decreasing), *then* by price (increasing)!

Restaurant	★	\$
Aloha Eats	4	2
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
Maison Parisienne	4	3
Fons Empanadas	5	2
Del Seoul	4.5	2
The Weiner's Circle	3.5	1



Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
<i>Cafe Luigi</i>	<i>4</i>	<i>1</i>
<i>Aloha Eats</i>	<i>4</i>	<i>2</i>
<i>Maison Parisienne</i>	<i>4</i>	<i>3</i>
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1

First attempt: a comparison function that looks at both

- Need custom function for each combination of criteria
 - ▶ Nightmare with user-selectable sorting criteria!
(e.g., sorting arrows on columns on web sites)
- Gets unwieldly really quickly with more than two criteria
- We need something better

Sorting Using Multiple Criteria

What if we want a good meal, but as cheaply as possible?

- Order by rating (decreasing), *then* by price (increasing)!

Restaurant	★	\$
Aloha Eats	4	2
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
Maison Parisienne	4	3
Fons Empanadas	5	2
Del Seoul	4.5	2
The Weiner's Circle	3.5	1



Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
<i>Cafe Luigi</i>	<i>4</i>	<i>1</i>
<i>Aloha Eats</i>	<i>4</i>	<i>2</i>
<i>Maison Parisienne</i>	<i>4</i>	<i>3</i>
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1

Second attempt: sort multiple times, once for each criterion

- Done naïvely, each pass will undo the work of the previous. No good.
- But if we preserve *some* of the ordering between passes...

Preserving Input Ordering

For one input, there may be many possible valid, sorted outputs:

Restaurant	★	\$
Aloha Eats	4	2
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
Maison Parisienne	4	3
Fons Empanadas	5	2
Del Seoul	4.5	2
The Weiner's Circle	3.5	1



Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
Aloha Eats	4	2
Cafe Luigi	4	1
Maison Parisienne	4	3
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1

Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
Maison Parisienne	4	3
Cafe Luigi	4	1
Aloha Eats	4	2
The Weiner's Circle	3.5	1
Angela's Burrito Style	3.5	1

Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
Maison Parisienne	4	3
Aloha Eats	4	2
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1

The input is already sorted from South to North.

A sorting algorithm which preserves input ordering for elements that sort the same is called a *stable* sort. Only 1st one is!

Which Sorts are Stable?

Of the ones we have seen / will see:

Stable

- Merge sort (yet another reason to like it)
 - ▶ Need a slightly different version than the one we saw.
See code on Canvas.
- Insertion sort
- Counting sort (bonus)

Unstable

- Selection sort
- Quicksort (soon)
- Heapsort (later in the quarter)

Radix Sort

Our solution for sorting using multiple criteria!

Idea:

- Start by sorting by the *least important* criterion
 - ▶ In our example, price
- Then successively sort by the other criteria in increasing order of importance, ending with the *most important* one
 - ▶ In our example, stars
- By using a *stable* sort throughout, the work done by earlier sorting passes is preserved!

History: Herman Hollerith, 1887

- Used by tabulating machines for the 1890 US census
- Only later (1954!) implemented on a computer

Radix Sort Example

Step 1: Sort by price

Restaurant	★	\$
Aloha Eats	4	2
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
Maison Parisienne	4	3
Fons Empanadas	5	2
Del Seoul	4.5	2
The Weiner's Circle	3.5	1



Restaurant	★	\$
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1
Aloha Eats	4	2
Fons Empanadas	5	2
Del Seoul	4.5	2
Maison Parisienne	4	3

Radix Sort Example

Step 2: Sort by stars, using a stable sort

Restaurant	★	\$
Cafe Luigi	4	1
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1
Aloha Eats	4	2
Fons Empanadas	5	2
Del Seoul	4.5	2
Maison Parisienne	4	3



Restaurant	★	\$
Fons Empanadas	5	2
Del Seoul	4.5	2
<i>Cafe Luigi</i>	<i>4</i>	<i>1</i>
<i>Aloha Eats</i>	<i>4</i>	<i>2</i>
<i>Maison Parisienne</i>	<i>4</i>	<i>3</i>
Angela's Burrito Style	3.5	1
The Weiner's Circle	3.5	1

By using a stable sort, the fact that cheaper restaurants were earlier in the table is preserved → price becomes the tiebreaker!

If we had more criteria, we would just do more passes!

Radix Sort

```
def radix_sort (v, comparators):  
    for cmp in comparators:  
        any_stable_in_place_sort(v, cmp)
```

Takes a vector of comparison functions, in order from least to most significant.

Can be easily adapted to use out-of-place sorts as well.

With k comparators and a $\mathcal{O}(n \log n)$ helper sorting function, complexity is $\mathcal{O}(k \times n \log n)$

Stop! Question time!

Before we move on to quicksort...

Anything unclear so far?

Anything I missed?

Quicksort: the Classic Sorting Algorithm

Developed by Tony Hoare in 1959

- The same Tony Hoare as Hoare triples! (laws, lecture 3)

Probably the most commonly used sorting algorithm in practice

- $\mathcal{O}(n \log n)$
 - ▶ But with a caveat; see later
- But better constant factors than merge sort
 - ▶ Matters in practice!
- Can do its work in-place

Quicksort: the Classic Sorting Algorithm

Developed by Tony Hoare in 1959

- The same Tony Hoare as Hoare triples! (laws, lecture 3)

Probably the most commonly used sorting algorithm in practice

- $\mathcal{O}(n \log n)$
 - ▶ But with a caveat; see later
- But better constant factors than merge sort
 - ▶ Matters in practice!
- Can do its work in-place

The original, in-place version is pretty gnarly.

- So let's start with a simpler version

Quicksort (simple version)

(Vector notation for clarity; works even better with linked lists!)

4	7	8	5	2	1	9	5	3
---	---	---	---	---	---	---	---	---

Quicksort (simple version)

(Vector notation for clarity; works even better with linked lists!)

4	7	8	5	2	1	9	5	3
---	---	---	---	---	---	---	---	---

Step 1: Choose one element as the *pivot*; here, 1st elt.

Quicksort (simple version)

(Vector notation for clarity; works even better with linked lists!)

4	7	8	5	2	1	9	5	3
---	---	---	---	---	---	---	---	---

Step 1: Choose one element as the *pivot*; here, 1st elt.

2	1	3
---	---	---

4

7	8	5	9	5
---	---	---	---	---

Step 2: Partition the list: elts $<$ pivot, and elts $\geq$ pivot.

Quicksort (simple version)

(Vector notation for clarity; works even better with linked lists!)

4	7	8	5	2	1	9	5	3
---	---	---	---	---	---	---	---	---

Step 1: Choose one element as the *pivot*; here, 1st elt.

1	2	3
---	---	---

4

5	5	7	8	9
---	---	---	---	---

Step 2: Partition the list: elts $<$ pivot, and elts $\geq$ pivot.

Step 3: Sort each sub-list recursively.

Quicksort (simple version)

(Vector notation for clarity; works even better with linked lists!)

4	7	8	5	2	1	9	5	3
---	---	---	---	---	---	---	---	---

Step 1: Choose one element as the *pivot*; here, 1st elt.

1	2	3
---	---	---

4

5	5	7	8	9
---	---	---	---	---

Step 2: Partition the list: elts $<$ pivot, and elts $\geq$ pivot.

Step 3: Sort each sub-list recursively.

1	2	3	4	5	5	7	8	9
---	---	---	---	---	---	---	---	---

Step 4: Append them back together.

Quicksort (simple version)

```
def quicksort(lst):  
    # Base cases  
    if lst is None or lst.next is None: return lst  
    # Step 1: choose a pivot  
    let pivot = lst.data # first element  
    # Step 2: partition the list  
    let below = filter(lambda x: x < pivot, lst.next)  
    let above = filter(lambda x: x >= pivot, lst.next)  
    # Step 3: sort each sublist recursively  
    below = quicksort(below)  
    above = quicksort(above)  
    # Step 4: append them back together  
    return append(below, cons(pivot, above))
```

Quicksort (simple version)

```
def quicksort(lst):  
    # Base cases  
    if lst is None or lst.next is None: return lst  
    # Step 1: choose a pivot  
    let pivot = lst.data # first element  
    # Step 2: partition the list  
    let below = filter(lambda x: x < pivot, lst.next)  
    let above = filter(lambda x: x >= pivot, lst.next)  
    # Step 3: sort each sublist recursively  
    below = quicksort(below)  
    above = quicksort(above)  
    # Step 4: append them back together  
    return append(below, cons(pivot, above))
```

Cool, what's the complexity?

filter, append $\mathcal{O}(n)$ (traverse list), recur on both sub-lists.

Feels a lot like merge sort, so $\mathcal{O}(n \log n)$ again?

Quicksort (simple version)

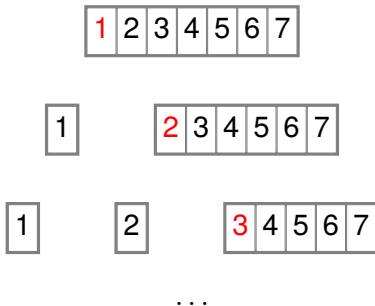
```
def quicksort(lst):  
    # Base cases  
    if lst is None or lst.next is None: return lst  
    # Step 1: choose a pivot  
    let pivot = lst.data # first element  
    # Step 2: partition the list  
    let below = filter(lambda x: x < pivot, lst.next)  
    let above = filter(lambda x: x >= pivot, lst.next)  
    # Step 3: sort each sublist recursively  
    below = quicksort(below)  
    above = quicksort(above)  
    # Step 4: append them back together  
    return append(below, cons(pivot, above))
```

Cool, what's the complexity?

filter, append $\mathcal{O}(n)$ (traverse list), recur on both sub-lists.
Feels a lot like merge sort, so $\mathcal{O}(n \log n)$ again?

Q: Under what assumptions, though?

Quicksort Complexity

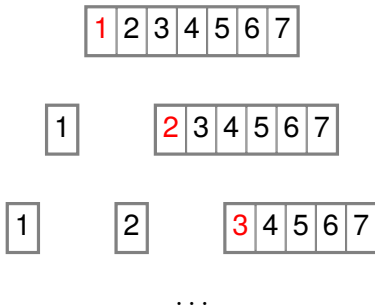


Every element is in the same sub-list!

Partition into sub-lists: $\mathcal{O}(n)$

Need to do it n times! $\rightarrow \mathcal{O}(n^2)$

Quicksort Complexity



Every element is in the same sub-list!

Partition into sub-lists: $\mathcal{O}(n)$

Need to do it n times! $\rightarrow \mathcal{O}(n^2)$

Can we do better by choosing a different element as pivot?

Quicksort Complexity

Bad news: no matter how we choose the pivot, some inputs will trigger the worst case ($\mathcal{O}(n^2)$)

- I.e., an attacker can create worst case inputs

Quicksort Complexity

Bad news: no matter how we choose the pivot, some inputs will trigger the worst case ($\mathcal{O}(n^2)$)

- I.e., an attacker can create worst case inputs

Compromise: have the worst case be ...

- ... uncommon in practice (i.e., not an already sorted list)
 - ▶ E.g., middle element pivot, median-of-three pivot, etc.
- ... difficult/impossible to guess on purpose
 - ▶ I.e., pick the pivot element at random (used in practice)

Quicksort Complexity

Bad news: no matter how we choose the pivot, some inputs will trigger the worst case ($\mathcal{O}(n^2)$)

- I.e., an attacker can create worst case inputs

Compromise: have the worst case be ...

- ... uncommon in practice (i.e., not an already sorted list)
 - ▶ E.g., middle element pivot, median-of-three pivot, etc.
- ... difficult/impossible to guess on purpose
 - ▶ I.e., pick the pivot element at random (used in practice)

Good news: except in these (rare) pathological cases, an *even enough* split between sub-lists $\rightarrow \mathcal{O}(n \log n)$

- Even enough $\rightarrow$ see later this quarter
- Quicksort is $\mathcal{O}(n \log n)$ in the *average case*
 - ▶ But still $\mathcal{O}(n^2)$ in the *worst case*
 - ▶ Overall, a good choice in practice

Quicksort Complexity

Bad news: no matter how we choose the pivot, some inputs will trigger the worst case ($\mathcal{O}(n^2)$)

- I.e., an attacker can create worst case inputs

Compromise: have the worst case be ...

- ... uncommon in practice (i.e., not an already sorted list)
 - ▶ E.g., middle element pivot, median-of-three pivot, etc.
- ... difficult/impossible to guess on purpose
 - ▶ I.e., pick the pivot element at random (used in practice)

Good news: except in these (rare) pathological cases, an *even enough* split between sub-lists $\rightarrow \mathcal{O}(n \log n)$

- Even enough $\rightarrow$ see later this quarter
- Quicksort is $\mathcal{O}(n \log n)$ in the *average case*
 - ▶ But still $\mathcal{O}(n^2)$ in the *worst case*
 - ▶ Overall, a good choice in practice

But why use it over merge sort? ($\mathcal{O}(n \log n)$, even worst case!)

- If you need in-place, merge sort can't but quicksort can

In-Place Quicksort

Same basic idea as the simple version

- Use one element as the pivot
- Partition other elements according to $<$ or $\geq$ pivot
- Sort each subpart recursively

But, doing the partition in-place is tricky!

- Can't just create new, separate sub-lists!
- Instead, need to swap elements so that everything to the left of the pivot is $<$, m.m. right and $\geq$
- After recursive sort, everything is in the right place!
 - ▶ No need to explicitly append!

In-Place Quicksort

4	7	8	5	2	1	9	5	3
---	---	---	---	---	---	---	---	---

Let's do one round of partitioning.

In-Place Quicksort

i j

4	7	8	5	2	1	9	5	3
---	---	---	---	---	---	---	---	---

First element is the pivot.

To use a random element as pivot, pick a random index and swap with first element. Then continue as follows.

Two indices that start just past the end: i and j .

In-Place Quicksort

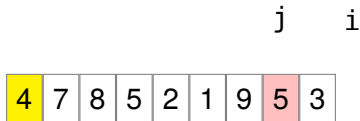
j i

4	7	8	5	2	1	9	5	3
---	---	---	---	---	---	---	---	---

Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement j and swap $v[i]$ and $v[j]$.

In-Place Quicksort

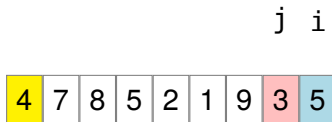
j i

4	7	8	5	2	1	9	5	3
---	---	---	---	---	---	---	---	---

Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

In-Place Quicksort

j i

4	7	8	5	2	1	9	3	5
---	---	---	---	---	---	---	---	---

Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

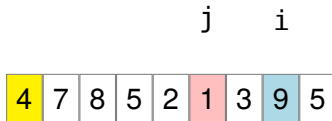
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

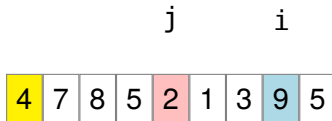
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

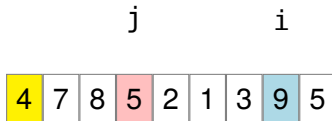
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

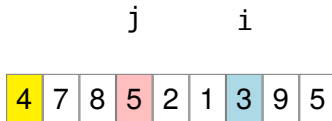
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

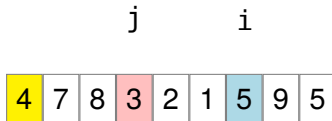
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

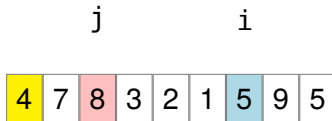
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

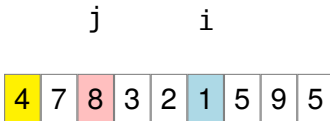
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

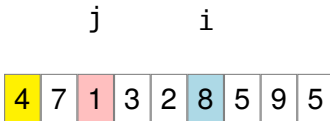
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

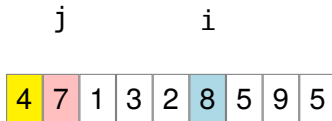
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

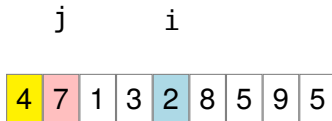
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

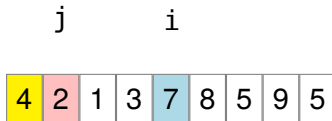
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

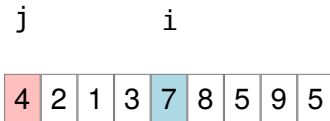
In-Place Quicksort



Decrement j

If $v[j] \geq \text{pivot}$, decrement i and swap $v[i]$ and $v[j]$.

In-Place Quicksort



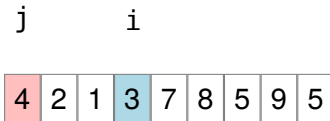
When we reach the pivot, swap it to its final position.

Everything to its left is $<$, m.m. right and $\geq$. Partition is done.

Doing one round of partitioning: $\mathcal{O}(n)$

- Intuition: Need to traverse vector back-to-front.

In-Place Quicksort



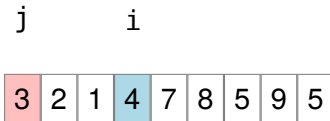
When we reach the pivot, swap it to its final position.

Everything to its left is $<$, m.m. right and $\geq$. Partition is done.

Doing one round of partitioning: $\mathcal{O}(n)$

- Intuition: Need to traverse vector back-to-front.

In-Place Quicksort



When we reach the pivot, swap it to its final position.

Everything to its left is $<$, m.m. right and $\geq$. Partition is done.

Doing one round of partitioning: $\mathcal{O}(n)$

- Intuition: Need to traverse vector back-to-front.

In-Place Quicksort

3	2	1	4	7	8	5	9	5
---	---	---	---	---	---	---	---	---

When we reach the pivot, swap it to its final position.

Everything to its left is $<$, m.m. right and $\geq$. Partition is done.

Doing one round of partitioning: $\mathcal{O}(n)$

- Intuition: Need to traverse vector back-to-front.

Recursively sort both sides and we're done!

In-Place Quicksort

1	2	3	4	5	5	7	8	9
---	---	---	---	---	---	---	---	---

When we reach the pivot, swap it to its final position.

Everything to its left is $<$, m.m. right and $\geq$. Partition is done.

Doing one round of partitioning: $\mathcal{O}(n)$

- Intuition: Need to traverse vector back-to-front.

Recursively sort both sides and we're done!

In-Place Quicksort

```
def in_place_quick_sort (v):  
    # Sort sub-array in range [lo,hi].  
    # Recur on sub-ranges, like binary search does.  
    def helper (lo, hi):  
        # Base case: length 1 or 0.  
        if hi-lo == 0 or hi-lo == -1:  
            return  
        let pivot = partition(v, lo, hi)  
        helper(lo, pivot-1)  
        helper(pivot+1, hi)  
    helper(0, v.len()-1)
```

Same pattern as simple version:

- $\mathcal{O}(n)$ work (partition)
- Recur on both subparts

Therefore, same complexity: $\mathcal{O}(n \log n)$ average, $\mathcal{O}(n^2)$ worst

In-Place Quicksort

```
# Partition sub-array [lo,hi].
def partition(v, lo, hi):
    let pivot = v[lo]
    let i = hi+1
    for j in range(hi, lo, -1):
        if v[j] >= pivot:
            i = i-1
            swap(v, i, j)
    # Move pivot to its right place.
    i = i-1
    swap(v, i, lo)
    return i
```

TL;DR: It's nasty and finicky, beware of off-by-one errors.
(Like any code dealing with array index math, really...)

Recap and More Resources

	Complexity	Stable?	In-Place?
Selection sort	$\mathcal{O}(n^2)$		
Merge sort	$\mathcal{O}(n \log n)$	✓	
Insertion sort	$\mathcal{O}(n^2)$	✓	✓
Radix sort	$\mathcal{O}(k \times n \log n)$	✓	✓
Quicksort	$\mathcal{O}(n \log n)^*$		✓
Counting sort	$\mathcal{O}(n)$	✓	

- Wikipedia has a nice table comparing different algorithms
 - ▶ https://en.wikipedia.org/wiki/Sorting_algorithm#Comparison_of_algorithms
- Galles's sorts are a bit different from ours, but same idea
 - ▶ <https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>
- The DSSL2 code for today's sorts is on Canvas
 - ▶ Try them out on different inputs!
 - ▶ Add print statements to see how they work!

Next time: Dictionary ADT

Bonus Sorting Topics

Is $\mathcal{O}(n \log n)$ the best we can do?

For sorts that rely on *comparisons* (i.e., $a < b$), **yes**.

But if our elements are drawn from a fixed set, we can sort using *counting* instead!

- Which makes it possible to sort in $\mathcal{O}(n)$!

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	0	0	0	0

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	0	0	1	0

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	0	0	2	0

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	0	0	3	0

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	1	0	3	0

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	1	0	3	1

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	2	0	3	1

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	2	0	3	2

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	2	0	3	3

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	3	0	3	3

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	3	0	4	3

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	3	0	4	3

Then construct the sorted output from the counts!

--	--	--	--	--	--	--	--	--	--

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	3	0	4	3

Then construct the sorted output from the counts!

--	--	--	--	--	--	--	--	--	--

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	3	0	4	3

Then construct the sorted output from the counts!

1	1	1							
---	---	---	--	--	--	--	--	--	--

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	3	0	4	3

Then construct the sorted output from the counts!

1	1	1							
---	---	---	--	--	--	--	--	--	--

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	3	0	4	3

Then construct the sorted output from the counts!

1	1	1	3	3	3	3			
---	---	---	---	---	---	---	--	--	--

Counting Sort Example

Want to sort a vector containing integers $\in [0, 4]$

3	3	3	1	4	1	4	4	1	3
---	---	---	---	---	---	---	---	---	---

5 possible elements, let's count how many of each we have.

0	1	2	3	4
0	3	0	4	3

Then construct the sorted output from the counts!

1	1	1	3	3	3	3	4	4	4
---	---	---	---	---	---	---	---	---	---

And voilà! Sorted. All we had to do was:

- Traverse the input vector: $\mathcal{O}(n)$
- Traverse the count vector: $\mathcal{O}(k)$, k usually $\lll n$
- Traverse the output vector: $\mathcal{O}(n)$
- **Total:** $\mathcal{O}(n)$

Counting Sort Code

```
# Sorts a vector of natural numbers below `bound`
def counting_sort (v, bound):
    let counts = [0; bound]
    for elt in v:
        counts[elt] = counts[elt] + 1
    let i = 0
    # less obviously O(n), but all the `j`'s add up to n
    for elt in range(counts.len()):
        for j in range(counts[elt]):
            # doing in-place, but could do out-of-place
            v[i] = elt
            i = i + 1
    return v
```

Bonus: can be adapted to other* fixed sets of possible elements, not just natural numbers $0 \dots n - 1$. See code on Canvas.

* certain conditions apply, void where prohibited.

Counting Sort Code

```
# Sorts a vector of natural numbers below `bound`
def counting_sort (v, bound):
    let counts = [0; bound]
    for elt in v:
        counts[elt] = counts[elt] + 1
    let i = 0
    # less obviously O(n), but all the `j`'s add up to n
    for elt in range(counts.len()):
        for j in range(counts[elt]):
            # doing in-place, but could do out-of-place
            v[i] = elt
            i = i + 1
    return v
```

If that's the best we can do, then what's the worst?

Comically Bad Sorts

Running joke: trying to come up with the worst possible sorting algorithm (that still works!).

- **Bogosort**

- ▶ Shuffle the list at random, check if it's sorted
 - ▶ If not, try again
- ▶ Not guaranteed to terminate, $\mathcal{O}(n!)$ in the average case
 - ▶ Could theoretically be $\mathcal{O}(n)$ on a quantum computer!

- **Slowsort**

- ▶ Parody of divide-and-conquer algorithms (Comp Sci 336)
- ▶ $\mathcal{O}(n^{(\log n)/2})$

- **Worstsort**

- ▶ Truly perverse, but guaranteed to terminate
- ▶ $\mathcal{O}((n!^{(f(n))})^2)$
where $n!^{(m)}$ means $((n!)!) \dots !$ (factorial iterated m times)
and $f(n)$ can be an arbitrarily bad function

- Etc.

Computer scientists have a strange sense of humor.

Helper Functions Analysis

Selection Sort Helpers (1/2)

```
def find_largest(lst):  
    if lst is None:  
        error('empty list has no largest element')  
    let best_so_far = lst.data  
    while lst is not None:  
        if lst.data > best_so_far:  
            best_so_far = lst.data  
        lst = lst.next  
    return best_so_far
```

Traverses the entire list; the largest element could be anywhere!

Thus, $\mathcal{O}(n)$.

Selection Sort Helpers (2/2)

```
# returns a new list, but could also do an  
# in-place version  
def remove(elt, lst):  
    if lst is None:  
        error('element not found: %d', elt)  
    elif lst.data == elt:  
        return lst.next  
    else:  
        return cons(lst.data,  
                     remove(elt, lst.next))
```

In the worst case, we're removing the last element of the list.

So we may need to traverse the whole list.

Thus, $\mathcal{O}(n)$.

Merge Sort Helpers (1/2)

Split a list into two sublists, one with the elements with even indices, the other with odd indices.

0th element, 2nd, 4th, etc.

```
def evens(lst):  
    if lst is None: return None  
    else: return cons(lst.data, odds(lst.next))
```

1st element, 3rd, 5th, etc.

```
def odds(lst):  
    if lst is None: return None  
    else: return evens(lst.next)
```

The mutual recursion may obscure matters a bit, but in the end, calling either helper traverses the entire list.

They are both $\mathcal{O}(n)$.

Merge Sort Helpers (2/2)

Merge two sorted lists into one, larger sorted list.

```
# : List[Number] List[Number] -> List[Number]
def merge(l1, l2):
    if l1 is None:
        return l2
    elif l2 is None:
        return l1
    elif l1.data < l2.data:
        return cons(l1.data, merge(l1.next, l2))
    else:
        return cons(l2.data, merge(l1, l2.next))
```

In merge sort, the inputs are both lists of length $\frac{n}{2}$.

merge traverses both lists all the way to the end, for a total length of n .

Thus merge is $\mathcal{O}(n)$.

Simple Quicksort Helpers

```
def filter(f, lst):  
    if lst is None:  
        return None  
    if f(lst.data):  
        return cons(lst.data, filter(f, lst.next))  
    else:  
        return filter(f, lst.next)
```

Intuition: traverses list front to back, builds a new one of the same length or shorter $\rightarrow \mathcal{O}(n)$. (Assuming f takes $\mathcal{O}(1)$ time.)

```
def append(lst1, lst2):  
    if lst1 is None: return lst2  
    if lst2 is None: return lst1  
    else:  
        return cons(lst1.data, append(lst1.next, lst2))
```

Intuition: traverses lst front to back $\rightarrow \mathcal{O}(n)$.